

LogiTrack

Relational Algebra Translations

Introduction

This document contains the Extended Relational Algebra translations for the six LogiTrack analytical SQL queries. The queries have been labeled with word descriptions to explain the logic step-by-step. The following extended operators are used:

- σ (**Sigma**): Selection (WHERE / HAVING)
 - π (**Pi**): Projection (SELECT)
 - $\bowtie$ (**Bowtie**): Natural or Theta Join (JOIN)
 - γ (**Gamma**): Grouping and Aggregation (GROUP BY, COUNT, SUM, AVG)
 - τ (**Tau**): Sorting (ORDER BY)
-

Query 1: Top 5 Drivers (On-Time Deliveries)

Logic Description: Join the **drivers** and **deliveries** tables. Filter for perfect deliveries (delivered, 0 delay, actual date exists). Group the results by driver, aggregate to count on-time deliveries and calculate average duration. Sort the results and limit to the top 5.

$$\text{Limit}_5 \left(\tau_{\text{on_time DESC, avg_dur ASC}} \left(\pi_{\text{driver_id, driver_name, on_time, avg_dur}} \left(\begin{aligned} &\text{driver_id, first_name, last_name} \gamma_{\text{COUNT(*)} \rightarrow \text{on_time, AVG(Duration) \rightarrow \text{avg_dur}} \left(\right. \\ &\left. \left. \sigma_{\text{status='delivered'} \wedge \text{delay_minutes}=0 \wedge \text{actual_date} \neq \text{NULL}} (\text{Dl} \bowtie_{\text{Dl.driver_id} = \text{D.driver_id}} \text{D}) \right) \right) \right) \right) \end{aligned} \right)$$

Note: Duration represents the timestamp difference calculation.

Query 2: Warehouses Above Average Delay

Logic Description: First, calculate the global company-wide average delivery delay. Then, join **deliveries** to **warehouses**, group by the warehouse, and aggregate the average delay per warehouse. Finally, filter (using HAVING/Sigma) to keep only those warehouses where their average is strictly greater than the global average, and sort the results.

Step 1: Global Average

$$\text{GlobalAvg} = \gamma_{\text{AVG(delay_minutes)}}(\text{Deliveries})$$

Step 2: Main Query

$$\tau_{\text{avg_delay DESC}} \left(\sigma_{\text{avg_delay} > \text{GlobalAvg}} \left(\begin{aligned} &w.\text{id}, w.\text{name} \gamma_{\text{AVG(delay_minutes) \rightarrow \text{avg_delay, COUNT(*) \rightarrow \text{total}}} (\text{Dl} \bowtie_{\text{Dl.warehouse_id} = \text{W.warehouse_id}} \text{W}) \right) \right) \end{aligned} \right)$$

Query 3: Delayed Routes Outnumbering On-Time Routes

Logic Description: Join `deliveries`, `routes`, and `cities` (twice, for origin and destination). Evaluate conditional logic for delayed vs. on-time statuses. Group by the route and cities, summing the conditional flags. Filter for routes where the delayed count exceeds the on-time count.

Conditional Logic definitions:

$\text{Cond}_d = \text{CASE WHEN status} = \text{'delayed'} \text{ THEN } 1 \text{ ELSE } 0$

$\text{Cond}_o = \text{CASE WHEN status} = \text{'delivered'} \wedge \text{delay_minutes} = 0 \text{ THEN } 1 \text{ ELSE } 0$

Main Query:

$$\begin{aligned} & \tau_{\text{delayed_count}} \text{ DESC } \left(\sigma_{\text{delayed_count} > \text{on_time_count}} \left(\right. \right. \\ & \quad \left. \left. r.\text{id}, c.o.\text{name}, c.d.\text{name} \gamma \text{SUM}(\text{Cond}_d) \rightarrow \text{delayed_count}, \text{SUM}(\text{Cond}_o) \rightarrow \text{on_time_count} \left(\right. \right. \right. \\ & \quad \left. \left. \text{Deliveries} \bowtie \text{Routes} \bowtie_{c.o.\text{id}=r.\text{origin_id}} \text{Cities}_{\text{origin}} \bowtie_{c.d.\text{id}=r.\text{dest_id}} \text{Cities}_{\text{dest}} \right) \right) \end{aligned}$$

Query 4: Rank Cities by Volume (Window Function)

Logic Description: Join `shipments`, `warehouses`, and `cities`. Group by city to sum the total volume. Finally, project a window function ranking based on the summed volume and sort by this rank.

$$\begin{aligned} & \tau_{\text{volume_rank}} \text{ ASC } \left(\pi_{\text{city_id}, \text{city_name}, \text{total_volume}, \text{RANK}() \text{ OVER}(\dots) \rightarrow \text{volume_rank}} \left(\right. \right. \\ & \quad \left. \left. c.\text{city_id}, c.\text{city_name} \gamma \text{SUM}(\text{volume_m3}) \rightarrow \text{total_volume} \left(\text{Shipments} \bowtie \text{Warehouses} \bowtie \text{Cities} \right) \right) \right) \end{aligned}$$

Query 5: Classify Deliveries by Performance

Logic Description: Project a new column (`PerfCat`) evaluating the `CASE WHEN` performance logic. Group the data by this newly projected category, aggregate the counts and averages, and sort by the highest count.

$$\begin{aligned} & \tau_{\text{delivery_count}} \text{ DESC } \left(\right. \\ & \quad \left. \text{PerfCat} \gamma \text{COUNT}(\ast) \rightarrow \text{delivery_count}, \text{AVG}(\text{delay_minutes}) \rightarrow \text{avg_delay} \left(\right. \right. \\ & \quad \left. \left. \pi_{\text{CASE}(\dots) \rightarrow \text{PerfCat}, \text{delay_minutes}}(\text{Deliveries}) \right) \right) \end{aligned}$$

Query 6: Simple Range Filter on Deliveries

Logic Description: Filter the `deliveries` table using equality and range parameters (warehouse ID and pickup date). Project the specified columns. No grouping or sorting is required.

$$\pi_{\text{delivery_id, shipment_id, driver_id, status, delay_minutes, actual_date}} \left(\sigma_{\text{warehouse_id}=1 \wedge \text{pickup_date} \geq '2025-09-01' \wedge \text{pickup_date} < '2026-01-01'} (\text{Deliveries}) \right)$$